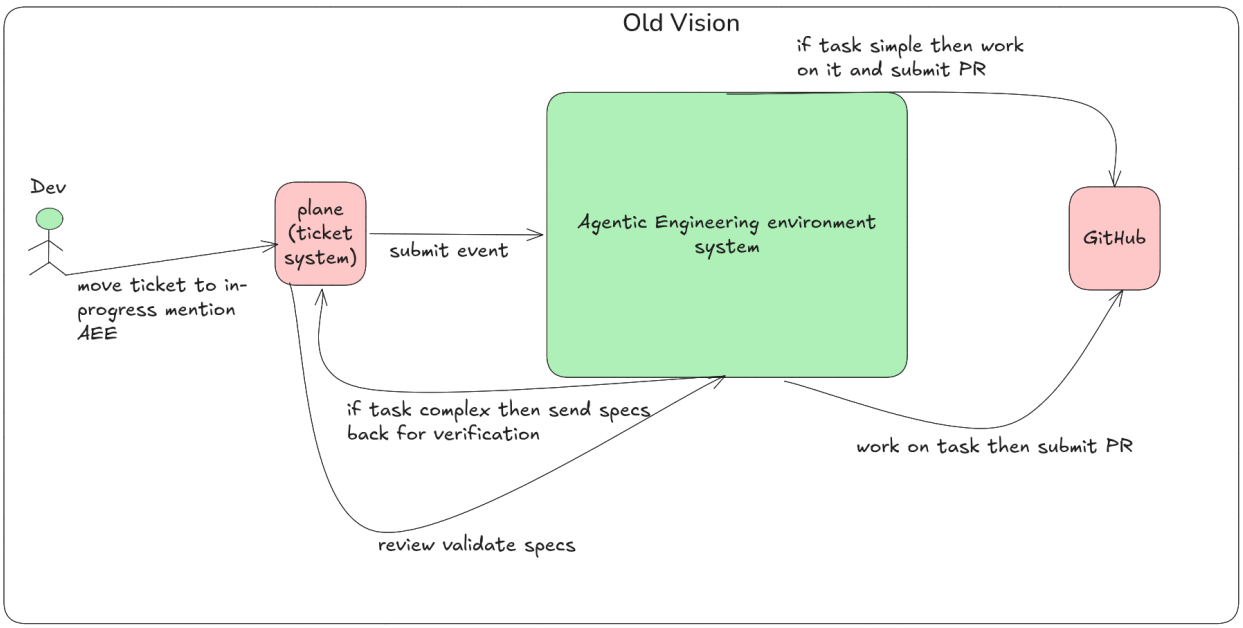
Old view:



1. Executive Summary: The Autonomous Engineering Environment (AEE)

The objective of the AEE is to transform isolated open-source AI utilities into a secure, institutional **Software Assembly Line** for Séminaire.com and Unitalk.ai. Rather than relying on rigid, hardcoded scripts, the AEE acts as a central nervous system that coordinates planning, execution, context retrieval, and safety validation into a unified enterprise utility.

The Problem

Evolving open-source tools (Hermes Agent, SpecKit, OpenCode) provide excellent individual capabilities but lack cohesive integration. They do not natively handle persistent state management, enterprise safety boundaries, multi-step human review cycles, or cross-tool data translation. Attempting to stitch them together directly creates brittle pipelines prone to context drift and infinite loops.

The Solution

A custom, **tool-agnostic AI orchestration harness** developed as a headless backend application. This platform abstracts the underlying AI engines behind stable interfaces, ensuring that the entire environment remains highly reliable, transparent, and completely insulated from open-source tool churn.

[Plane/Unitalk Event] ⟶ [AEE Harness (Brain + RAG)] ⟶ [Sandboxed Hermes Worker] ⟶ [Verified PR]

Core Strategic Pillars

- **Decoupled Architecture:** Built using a modular ports-and-adapters paradigm. Third-party components (e.g., Plane, LanceDB, OpenCode) act as pluggable modules. If a tool changes or is swapped in the future, the core proprietary orchestration logic remains untouched.
- **Native Interface UX (Zero-Dashboard Overhead):** Developers and product managers collaborate with the system using existing workflows. Moving a ticket triggers the engine, adjustments are handled via plain-text feedback loops, and human-in-the-loop (HITL) checkpoints are unlocked natively with simple commands (/approve).
- **Brownfield-First Optimization:** The 8-week MVP targets existing codebases with established architectural patterns and active test suites. This operational boundary eliminates non-deterministic greenfield configuration traps, leveraging pattern mimicry to deliver immediate, measurable development velocity safely.

2. The Decoupled Component Framework (Core Architecture)

The AEE is designed around a **Ports-and-Adapters (Hexagonal) Architecture**. The core orchestration engine relies entirely on abstract interfaces. Third-party frameworks and APIs are treated merely as interchangeable plugins (Adapters). This guarantees that a change in any single open-source tool will not cause a cascading failure across the rest of the ecosystem.

2.1 Planning & Specification Layer

- **The Port (Interface):** Ingests raw issue descriptions and project metadata; outputs a standardized, deterministic technical specification markdown document (SpecificationDocument).
- **Current Adapter: SpecKit** (Spec-Driven Development runner).
- **Decoupled Strategy:** If SpecKit is swapped out, the core engine simply points the interface to a direct LLM routing prompt (e.g., Claude 3.5 Sonnet) trained on the same markdown schema.

2.2 Knowledge Base Layer (Code Intelligence)

- **The Port (Interface):** Synchronizes local repository file trees into semantic vectors; processes natural language queries from agents to return high-relevance code snippets, complete with file paths and line ranges.
- **Current Adapter: LanceDB** (embedded vector database native to Hermes).
- **Decoupled Strategy:** The interface isolates the vector operations. Swapping LanceDB for Qdrant, ChromaDB, or an AST-based code map tool (like Aider) requires rewriting only the database ingestion and retrieval client logic.

2.3 External Context Subsystem

- **The Port (Interface):** Intercepts queries regarding external libraries; fetches fresh web search results and version-correct, official API definitions to eliminate LLM hallucinations.
- **Current Adapters: Context7 MCP** (Model Context Protocol for documentation) & **Tavily Search** (web lookup).
- **Decoupled Strategy:** The core engine interacts with a generic ExternalInformationProvider. If Context7 is replaced by a custom documentation scraper, the downstream agent loop remains completely unchanged.

2.4 Tooling & Skills Registry

- **The Port (Interface):** Declares and registers specific function definitions (e.g., read_file, execute_bash) that agents are permitted to use. It validates payloads and acts as the runtime execution router.
- **Current Adapter: Hermes Skills System** (Nous Research skill encapsulation).
- **Decoupled Strategy:** Tool calling is normalized to JSON schemas. If the system shifts away from Hermes' native skills, the tool registry can map directly to standard OpenAI

tool-calling definitions or LangChain function decorators.

2.5 Execution Sandbox

- **The Port (Interface):** Spawns an isolated, temporary environment with a localized file tree clone; executes file system alterations and shell operations; returns standard output, errors, and system diffs.
- **Current Adapter: OpenCode / Kilocode** Docker instances running on a private Linux server.
- **Decoupled Strategy:** The core engine executes commands against an abstract container layer. Swapping OpenCode for native Git Worktrees (OpenKit) or secure micro-VMs (like E2B) requires updating only the workspace provisioner class.

2.6 Memory & State Ledger

- **The Port (Interface):** Acts as the centralized state machine for long-running workflows. It persists agent loop histories, task execution tracking, retry counters, and system-wide state markers (e.g., Blocked, Running, Completed).
- **Current Adapters: Plane API** (project management webhook plane) & **Convex** (real-time state synchronization).
- **Decoupled Strategy:** The state machine runs independently of the visual board. If Plane is swapped for Linear, or Convex is replaced by a PostgreSQL database with a Redis queue, the background orchestration transitions remain identical.

2.7 Quality Assurance (QA) Layer

- **The Port (Interface):** Programmatically validates the generated code inside the sandbox environment. Automatically executes unit tests, code linters, compilation steps, and UI regressions, returning a definitive binary Pass/Fail result.
- **Current Adapter:** Native repository test runners combined with **Browser-use / Playwright** for headless, agent-driven visual testing and UI layout verification.
- **Decoupled Strategy:** Code verification relies entirely on execution logs and test suite exit codes. Whether visual regression is tested via local browser runtimes or third-party cloud testing grids, the application harness receives the same structured results payload.

2.8 Observability Layer

- **The Port (Interface):** Traces and logs all systemic AI telemetry, monitoring real-time

token utilization, inferencing latencies, comprehensive agent prompt-response chains, and ongoing API financial metrics.
- **Current Adapter: Langfuse** (open-source LLM engineering platform).
- **Decoupled Strategy:** Telemetry ingestion hooks run on open standards. Switching from Langfuse to alternative logging platforms like Phoenix (Arize) or Prometheus/Grafana requires updating only the outbound telemetry middleware.

3. The Tool-Agnostic Principle (Future-Proofing against OS Churn)

The open-source AI and developer-tooling ecosystem changes at a volatile pace. New models, sandboxing strategies, and agent frameworks emerge weekly. To safeguard the engineering equity of Séminaire.com and Unitalk.ai, the AEE enforces a strict **Tool-Agnostic Architecture**. The system's intelligence must never depend on the specific features or APIs of a third-party open-source component.

3.1 Architectural Isolation (Ports vs. Adapters)

The core application logic functions entirely inside a protected domain. It communicates with external systems through abstract definitions (Ports). The third-party tools are treated as disposable utilities plugged into these ports via specialized translation code (Adapters).

- **The Core Engine (Stable):** Contains the business rules, the multi-agent state machinery, context packaging algorithms, and token financial guardrails. It never shifts when external tools change.
- **The Integration Layers (Volatile):** Low-level code wrappers that translate third-party data formats (e.g., Plane APIs, Hermes CLI outputs, LanceDB queries) into formats the core engine natively understands.

3.2 Concrete Proof: Swapping Project Management Boards

If the enterprise decides to migrate its project workflows from Plane back to Linear, a tightly-coupled system would require an extensive, ground-up rewrite of the backend event

listeners. Under a tool-agnostic architecture, the transition is risk-insulated:

1. The underlying state ledger and background agent triggers remain completely untouched.
2. A new isolated adapter class (LinearProjectAdapter) is written to map Linear's incoming webhook payload to the core engine's existing ProjectManagementInterface.
3. The plugin is swapped at the configuration level, maintaining 100% execution uptime and continuity across the core agent factory.

4. The Developer Experience (UX) & Human-in-the-Loop (HITL) Workflow

To eliminate frontend development overhead and ensure immediate developer adoption, the AEE rejects custom visual dashboards. Instead, it embeds the Human-in-the-Loop (HITL) orchestration directly within the team's native workspace environments: **Plane** and **Forgejo**, driven by real-time webhooks and text-based slash commands.

4.1 The Core Workflow Lifecycle Matrix

Phase	System Action	User/Human Action
1. Trigger	Backend harness catches the Plane webhook or explicit agent @mention event.	Manager or developer moves a ticket to "In Progress" on Plane.
2. Spec Generation	SpecKit analyzes repository context via LanceDB, compiles a Markdown technical blueprint, and posts it as a thread comment. State is atomically set to Blocked.	None (Automatic Safety Brake). The engine halts execution until explicitly authorized.
3. Review	Orchestrator enters an idle polling state, listening for verified webhook payloads	Engineer reviews specification layout. Interacts natively via

Phase	System Action	User/Human Action
Loop	on the active ticket comment thread.	markdown text responses.
4. Execution	Hermes engine unblocks, provisions an OpenCode container sandbox, applies file mutations, executes test suites, and pushes a PR.	Review the finalized code change and unit/visual test execution logs directly inside Forgejo Git.

4.2 Concrete State Control Interfaces

Human authority over the background server engine is strictly enforced through two explicit text-driven interaction pathways on the Plane thread:

- **The Happy Path (Approval):** Typing /approve instantly triggers the harness to transition the background task status from Blocked to Running. This releases the safety brake and authorizes Hermes to initiate code operations inside the sandbox container.
- **The Pivot Path (Iterative Revision):** Typing @Hermes revise: [explicit directive] instructs the system to ingest the text loop. The engine feeds the adjustment back into the Planning Layer alongside original requirements, generating an updated **Specification V2** comment and maintaining the hard safety freeze until an explicit /approve signal is captured.

5. Project Anatomy Strategy: Brownfield vs. Greenfield Optimization

To guarantee predictable delivery within the aggressive 8-week MVP timeline, the AEE enforces a strict segregation based on codebase maturity. The environment is explicitly optimized 100% for **Brownfield (Existing) Repositories** rather than Greenfield (Brand New) applications.

5.1 Pattern Mimicry vs. Pure Invention

Large Language Models and agent frameworks operate optimally as advanced pattern-matching systems. Under a Brownfield environment, the codebase already possesses an established architecture:

- **The "Constitution" Advantage:** Pre-existing directory structures, naming conventions, linting rules, and dependency choices serve as an explicit template. Using LanceDB, agents extract existing files to securely mimic established code styles, preventing architectural drift.
- **The Greenfield Anti-Pattern:** Building from scratch requires the agent to make unguided, high-level choices regarding scaffolding, boilerplate, and package lock-files. Without historical patterns, agent behavior becomes highly non-deterministic, inevitably resulting in cascading compilation loops.

5.2 Deterministic Testing & Safe Boundaries

The main engine driving the QA sandbox is objective validation. Brownfield codebases bring pre-installed infrastructure constraints:

- **Binary Success Criteria:** Active unit, integration, and visual test suites give the sandbox runner a literal, automated definition of success. If the execution runner modifies code and the test suites exit with code 0, the pipeline passes.
- **MVP Target:** The initial 8-week build will lock its execution footprint onto a single, active, and well-tested Pilot Repository within Séminaire.com or Unitalk.ai. This provides immediate productivity gains for the live team while isolating infrastructure testing to controlled perimeters.

STRATEGIC SCOPE BRIEF

Autonomous Engineering Environment (AEE)

An Agentic Infrastructure Framework for Séminaire.com & Unitalk.ai

Prepared For:

Patrick Chassany CEO, Unitalk.ai & Co-Founder, Séminaire.com

Prepared By:

Zakaria Baqasse AI & Software Infrastructure Engineer

Date:

May 2026

CONFIDENTIALITY & PROPRIETARY NOTICE

This document contains proprietary scoping methodologies, evaluation frameworks, and strategic operational insights intended solely for the review of the leadership teams at Séminaire.com and Unitalk.ai. Dissemination, distribution, or copying of this document without explicit prior consent is strictly prohibited. The contents herein represent a functional specification framework; technical implementation architecture remains the intellectual property of the author until formal engagement terms are executed.

Table of Content:

1. Executive Summary & Core Mission (The "Why")

1.1 Project Vision

The objective of this initiative is to design and evaluate an **Autonomous Engineering Environment (AEE)**—a highly structured, multi-agent software delivery pipeline tailored for the engineering teams at Séminaire.com and Unitalk.ai.

Rather than treating AI as an isolated, interactive chat assistant for individual programmers, the AEE frames autonomous agents as scalable team members. The project aims to establish an end-to-end multi-agent workspace that transforms raw product ideas into production-ready Pull Requests with absolute predictability, rigorous validation gates, and minimal operational overhead.

1.2 The Core Problem: The Limits of "Vibe Coding"

As autonomous coding tools have evolved, engineering organizations have run into a systemic bottleneck: **the unpredictability of unstructured AI execution.** Traditional agent implementations rely heavily on a conversational chat paradigm, which presents severe operational challenges at scale:

- **Absence of Durable State:** Standard chat threads lack persistent state outside the immediate context window, making it impossible to coordinate multiple agents working on parallel features.
- **Context Drift & Hallucination:** Allowing agents to write code directly from loose user prompts often leads to fragmented implementations, broken dependencies, and massive technical debt.
- **Siloed Multi-Agent Execution:** Without strict workspace boundaries and centralized tracking, running parallel agent sessions results in code collisions, runtime pollution, and visibility gaps for engineering managers.

1.3 The Strategic Objective: The Software Assembly Line

The AEE solves these challenges by implementing a highly deterministic, **Spec-Driven Development (SDD)** lifecycle. Instead of jumping straight to code generation, the environment establishes a rigid hierarchy of execution:

[Repository Context] ⟶ [Executable Spec] ⟶ [Technical Plan] ⟶ [Isolated Task Implementation]

By centering the engineering lifecycle around verifiable markdown artifacts and utilizing an organization's existing project management interface as the core orchestrator, the system achieves three primary outcomes:

1. **The Issue Tracker as a Control Plane:** Shifting the agent steering mechanism out of chat windows and into project boards (e.g., Linear). Moving a ticket automatically orchestrates background agent behaviors, making the entire system auditable, visible, and deeply integrated into existing human engineering workflows.
2. **Deterministic Quality Gates:** Ensuring non-deterministic AI agents operate within predictable boundaries by enforcing strict spec-validation, automated compilation tests, linting routines, and mandatory human-in-the-loop review checkpoints before any code hits a codebase.
3. **True Workspace Isolation:** Unlocking safe, parallel feature development by dynamically provisioning partitioned sandboxes for separate tasks, ensuring multiple agents can execute changes simultaneously without data corruption or cross-contamination.

Ultimately, the core mission of the AEE is to decouple software scaling from pure headcount, enabling the core development teams to focus on architecture and high-level steering while autonomous agents absorb the heavy execution lift.

2. Target Functional Workflow (The "What")

The Autonomous Engineering Environment (AEE) maps out a strict, step-by-step pipeline designed to take an abstract business requirement and translate it into production-grade code. By separating this workflow into five distinct, sequential phases, the system ensures complete traceability, isolated execution, and predictable quality.

2.1 Overview of the End-to-End Pipeline

The functional boundary of the system is designed as an automated, closed-loop lifecycle that begins and ends inside the team's existing developer tooling (GitHub and Linear).

Phase	Input	Agentic Activity	Guardrail / Boundary	Output
1. Ingestion	Raw ticket / Intent	Context analysis & codebase mapping	Human-in-the-loop review	Executable Markdown Spec
2. Planning	Approved Spec	Task breakdown & dependency mapping	Rigid schema validation	Linear Tickets / Subtasks
3. Execution	Active Linear Ticket	Parallel coding in isolated runtimes	Zero cross-work space leakage	Feature Code
4. Verification	Unverified Code	Automated compilation, testing, & QA	Hard pass/fail criteria	Validated Build
5. Closure	Passing Sandbox Build	PR generation & status synchronization	Main branch protection	Mergeable Pull Request

2.2 Phase-by-Phase Breakdown

Phase 1: Ingestion & Specification Generation

- **The Workflow:** The process triggers when a new feature request or technical issue is introduced. A specialized Product/Architecture Agent ingests the current state of the relevant GitHub repository alongside the requirement context.
- **Agent Responsibility:** The agent reconciles the request against the existing codebase structure, identifies impacted files, and drafts a comprehensive, artifact-driven specification.
- **Boundary Control:** The agent does not write code. It generates a standardized, markdown-based specification file. This phase forces a mandatory **Human Checkpoint** where an engineering lead must review and digitally sign off on the specification before the pipeline can advance.

Phase 2: Technical Planning & Decomposition

- **The Workflow:** Once the specification is approved, the control plane shifts the context to a Planning Agent.
- **Agent Responsibility:** This agent evaluates the signed specification and splits the implementation work into granular, independent, and logically ordered technical tasks. It maps out dependencies between tasks to determine which features can be built in parallel and which require sequential execution.
- **Boundary Control:** The output of this phase is strictly behavioral and tracking-oriented. The planning agent interacts directly with the team's project management tool (Linear), automatically populating it with structured tickets, descriptions, and dependency links, establishing full visibility for management.

Phase 3: Parallel Execution & Workspace Isolation

- **The Workflow:** The moment a task ticket transitions to an "In Progress" state on the project board, the core execution harness initializes.
- **Agent Responsibility:** Dedicated coding agents are dispatched to execute individual tasks. To unlock true velocity, multiple agents work simultaneously on separate tickets belonging to the same broader feature branch.
- **Boundary Control:** To prevent agents from overriding each other's work or corrupting the main environment, each agent operates inside a completely **isolated, sandboxed workspace** (leveraging secure, virtualized runtimes). Agents are restricted to reading only the codebase state relevant to their ticket and executing safe, non-destructive file operations within their specific sandbox boundary.

Phase 4: Automated Verification Gates

- **The Workflow:** Code generation cannot exit the sandbox environment without proving its integrity. Once an execution agent completes its code modifications, it triggers an automated verification routine within its isolated workspace.

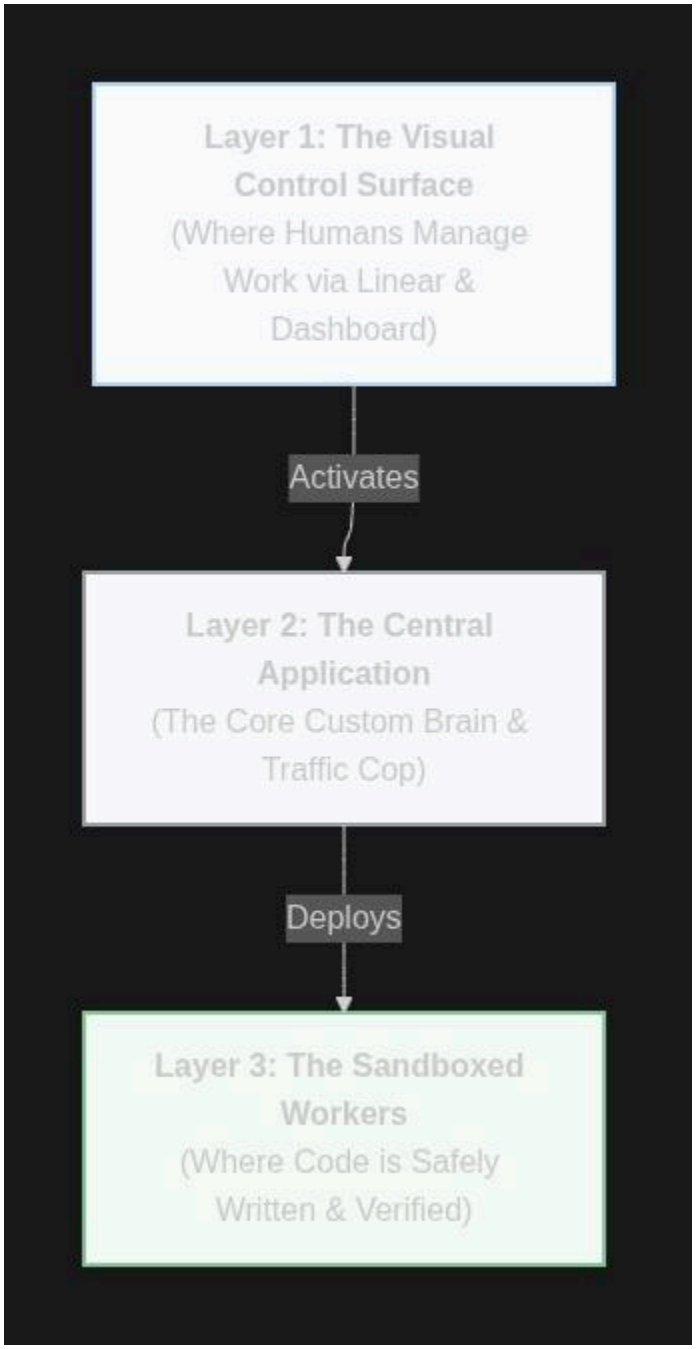
- **Agent Responsibility:** The verification runner triggers a series of deterministic engineering checks, including codebase compilation, project-specific linting configurations, local test suite execution, and basic static security analysis.
- **Boundary Control:** This is a zero-trust environment. If a test fails or a linter rejects the code formatting, the pipeline blocks progression. The error logs are routed back into the local agent's execution loop for self-correction. The code cannot leave the isolated sandbox until all verification gates return a deterministic "Pass" status.

Phase 5: Centralized Tracking & Merging

- **The Workflow:** Once a sandboxed environment successfully passes all verification gates, the AEE handles the final delivery phase.
- **Agent Responsibility:** The system aggregates the verified code changes, generates a cleanly formatted GitHub Pull Request containing a detailed log of the changes made, the test results, and the original ticket reference. Simultaneously, it updates the central project board, moving the corresponding Linear ticket to "Ready for Review."
- **Boundary Control:** The agent is completely restricted from merging code directly into protected production branches. The final merge action remains an absolute human privilege, ensuring that engineering teams maintain final oversight and strategic control over the codebase.

3. Platform Architecture & The Form-Factor (How It Works)

To make this a reliable asset for Séminaire.com and Unitalk.ai, we are not just running loose scripts. We are building a unified **Autonomous Engineering Application**. Think of open-source tools like engines, tires, and transmissions; our application is the actual vehicle that integrates them. The platform operates across three simple layers:



3.1 The Three-Layer Mapping

Layer 1: The Visual Control Surface (Management & Observability)

- **The Form Factor:** The user interface your team already uses, paired with a lightweight internal web dashboard.

- **The Tools Used: Linear** (Project Management) & **GitHub Pull Requests**.
- **How It Functions:** Patrick or an engineering lead moves a ticket to "In Progress" on Linear. That is the only human action required. The team doesn't look at complex terminal windows; they see agent progress updates right inside Linear, watch a live activity feed via the app's control center, and review the final output as a standard GitHub Pull Request.

Layer 2: The Central Application (The Custom Brain & Traffic Cop)

- **The Form Factor:** A centralized, long-running backend application (the custom glue we are building).
- **The Tools Used: OpenAI's Symphony** & **GitHub's Spec-Kit**.
- **How It Functions:** This is the core middleware. It listens to Linear webhooks using **Symphony** rules to track the state of a project. Once triggered, it runs **Spec-Kit** logic to automatically take a rough ticket description and map out a strict technical specification and task list. It acts as the central traffic cop, ensuring that arbitrary AI behavior is forced into a predictable software delivery pipeline.

Layer 3: The Sandboxed Workers (Secure Code Execution)

- **The Form Factor:** Isolated, automated background coding environments.
- **The Tools Used: OpenKit** & **Pi Agent**.
- **How It Functions:** For every single sub-task generated by Layer 2, the application uses **OpenKit** to spin up an isolated code workspace using native Git worktrees. This guarantees that multiple agents can write code simultaneously without overwriting each other. Inside each secure workspace, a lightweight **Pi Agent** execution harness handles the low-level work: reading the files, writing the changes, and running local test commands. Code is blocked from escaping this sandbox until all verification tests return a "Pass" status.

3.2 The Reality of the Development Effort

Building an institutional software factory is vastly different from using individual AI extensions. The core engineering effort lies in constructing the proprietary integration layer:

- **State Management:** Ensuring background agent processes can pause for hours while waiting for human specification approvals without dropping context or getting stuck in infinite loops.
- **Multi-Tenant Separation:** Ensuring the data, repository secrets, and logic models for Séminaire.com are rigidly partitioned from Unitalk.ai within the same centralized application.
- **Orchestration Logic:** Translating the unique open-source inputs and outputs of Spec-Kit and Pi Agent into a singular, predictable communication pipeline.

4. Tooling Landscape & Evaluation Matrix (The Candidate Stack)

To build a resilient Autonomous Engineering Environment (AEE) without succumbing to vendor lock-in or fragile custom scripts, the platform will leverage the modern, open-source agentic infrastructure layer.

Rather than adopting a single opinionated framework, our approach treats the ecosystem as a modular stack. The primary objective of the initial engineering phase is to audit, benchmark, and integrate four core candidate technologies.

4.1 The Architectural Layers & Candidate Technologies

1. Specification & Guardrail Layer: GitHub's Spec-Kit

- **Role in the Stack:** Enforces Spec-Driven Development (SDD) to eliminate unstructured "vibe-coding."
- **Core Mechanics:** Spec-Kit operates via a structured, artifact-driven pipeline that forces an agent to transition cleanly from a high-level description (`/specify`) to a rigorous technical blueprint (`/plan`) and finally into micro-deliverables (`/tasks`).
- **Evaluation Criteria:** We will evaluate its ability to successfully ingest complex, pre-existing multi-repo structures and assess how cleanly it enforces strict schema constraints onto downstream coding agents.

2. Control Plane & Orchestration Daemon: OpenAI's Symphony

- **Role in the Stack:** Serves as the long-running background scheduler that transforms the issue tracker into the execution engine.
- **Core Mechanics:** Symphony is an open-source, continuous orchestration service that reads a repository-level policy file (`WORKFLOW.md`), polls tracking tools like Linear for active issue states, manages agent concurrency queues, handles session retries, and pushes live status updates back to management surfaces.
- **Evaluation Criteria:** Benchmarking the stability of its long-running polling loop, tracking its token-overhead efficiency during state reconciliation, and auditing its failure-recovery protocols during API rate-limiting events.

3. Workspace Isolation & Git Middleware: OpenKit (openkit.work)

- **Role in the Stack:** Handles environment provisioning, preventing data leakage and runtime collisions between parallel workers.
- **Core Mechanics:** OpenKit interfaces with the local server layer to spin up completely isolated task spaces built on native **Git worktrees**. It provides a centralized configuration plane across multiple projects and features built-in, automated port conflict resolution (offsetting runtime ports dynamically so multiple agents can run independent test servers concurrently).
- **Evaluation Criteria:** Testing the performance of rapid worktree generation under high concurrency, measuring multi-tenant execution boundaries, and evaluating its node/runtime abstraction stability.

4. Code Execution Harness: Pi Agent (pi.dev)

- **Role in the Stack:** The minimalist, lightweight runner that performs the actual file edits and terminal operations.
- **Core Mechanics:** Created as a high-reliability alternative to bloated coding frameworks, Pi Agent utilizes an ultra-lean system prompt (under 1,000 tokens) and exposes only four primitive tools: `read`, `write`, `edit`, and `bash`. Its primary strength is its tree-structured JSONL session tracking (allowing the system to fork or rollback to earlier execution states if an agent breaks a build) and its hot-reloadable TypeScript SDK for runtime extensions.
- **Evaluation Criteria:** Measuring its baseline reliability across varied LLM providers (Anthropic, OpenAI, OpenRouter) and designing custom TypeScript extensions to securely gate raw shell execution.

4.2 Evaluation Matrix & Selection Criteria

The candidate stack will be audited during the initial proof-of-concept sprint against a strict performance matrix. A tool must achieve a passing grade across all four engineering vectors to be approved for production infrastructure.

Evaluation Vector	**Target Engineering Benchmark**	**Critical Risk Factor**
Deterministic State Control	The tool must guarantee clear state-tree checkpoints (`/tree` and `/fork` mechanics) so the system can gracefully roll back failed agent code loops without blowing out token budgets.	*State Drift:* Agents getting trapped in infinite, expensive loops attempting to self-correct a broken build.

Workspace Isolation	Dynamic task sandboxing must use native OS and Git primitives (like Git worktrees) to ensure absolute code segregation during parallel tasks.	*Race Conditions:* Parallel agents overriding or clashing on shared project files or overlapping dependencies.
Context Compactness	Core runner prompts must remain minimalist (targeting a sub-1,000 token baseline) to maximize available context window space for complex codebases.	*Context Exhaustion:* Feature descriptions or error logs choking the model, driving up inference latency and API costs.
Tooling Reusability	Setup and agent skills must live entirely within the repository configuration (`WORKFLOW.md` or local skill directories), decoupled from proprietary developer platforms.	*Vendor Lock-in:* Becoming dependent on closed, non-portable, user-scoped enterprise AI suites.

5. Target MVP Roadmap & Milestones (8-Week Bounded Infrastructure Timeline)

*To manage execution risk, eliminate scope creep, and guarantee delivery within an aggressive 8-week window, the development of the Autonomous Engineering Environment (AEE) is structured as an **infrastructure-first, headless MVP**.*
By ruthlessly optimizing for backend stability and utilizing existing engineering interfaces (Linear and GitHub) as the visual control surfaces, we eliminate the overhead of building custom web applications. This roadmap treats the platform as a high-reliability pipeline, verifying sandbox isolation and deterministic guardrails at a single-repo scale before expanding the footprint.

5.1 Milestone Breakdown

Milestone 1: Tooling Evaluation Report & Bounded Architecture Blueprint

- ***Timeline:** Weeks 1–2*
- ***Functional Deliverables:***
    - *Completion of localized technical spikes for GitHub's Spec-Kit, OpenKit, Pi Agent, and OpenAI's Symphony spec to audit real-world API limits and bugs.*
    - *A comprehensive Tooling Evaluation Matrix mapping out token efficiency, context retention boundaries, and worktree isolation performance.*
    - *A finalized **Bounded Architecture Blueprint** defining the integration data contracts for a single pilot repository, explicitly mapping hard-capped agent capabilities (restricting code execution to specified file directories and ticket types).*
- ***Strategic Value:** Locks down the core technology primitives and structural constraints under localized conditions, completely freezing the project scope before backend development begins.*

Milestone 2: Single-Threaded "Headless" Core Spike (The "Walking Skeleton")

- ***Timeline:** Weeks 3–5*
- ***Functional Deliverables:***
    - *A fully functioning, single-threaded backend orchestration engine running in a local environment.*
    - *Webhook automation hooks that successfully monitor a test Linear board, capture ticket state shifts, run Spec-Kit logic, and provision a singular isolated Git worktree via OpenKit.*

    - *Successful execution of local verification gates (compilation, linting, tests) inside the sandbox, culminating in a single agent pushing an automated Pull Request to GitHub.*
    - ***Zero Dashboard Dependency:*** *All human interactions, including specification approvals and error tracking, are handled directly through Linear ticket updates and GitHub markdown logs.*
- ***Strategic Value:*** *Proves end-to-end integration plumbing. It demonstrates that the background orchestration layers can seamlessly move code from an issue tracker to a remote branch without requiring a custom user interface.*

Milestone 3: Multi-Agent Concurrency Engine & Pilot Team Deployment

- ***Timeline:*** *Weeks 6–8*
- ***Functional Deliverables:***
    - *Scaling the backend application layer to handle parallel multi-agent worker queues within the single pilot repository.*
    - *Implementation of concurrent workspace boundary controls, ensuring multiple isolated Git worktrees run independent test suites simultaneously without race conditions or shared port collisions.*
    - *Production integration with the team's live Linear workflow, establishing* ***Human-in-the-Loop (HITL) checkpoints*** *triggered via standard platform comments (e.g., typing* /approve *to sign off on a spec or merge a sandbox build).*
    - *Final deployment of the core headless engine, configured for immediate active dogfooding by a single selected development team.*
- ***Strategic Value:*** *Delivers the full autonomous software assembly line within a strict, risk-contained perimeter, empowering the organization to scale development velocity while maintaining absolute oversight.*

5.2 Summary of Target Execution Windows

Milestone / Deliverable	***Estimated Completion Window***	***Scope Boundary & Dependency***	***Core Validation Metric***

1. Tool Evaluation & Policy Specs	*End of Week 2*	*Access to a target sample repository.*	*Verification of token-limit guardrails & schema constraints.*
2. Headless Local Spike	*End of Week 5*	*Completed Milestone 1. Fully interface-less (Linear/GitHub only).*	*100% automated transition from a single ticket to a verified, compiled PR.*

3. Multi-Agent Production MVP	*End of Week 8*	*Completed Milestone 2. Bounded to a single pilot team/repository.*	*Zero-collision on parallel task completion across isolated concurrent worktrees.*